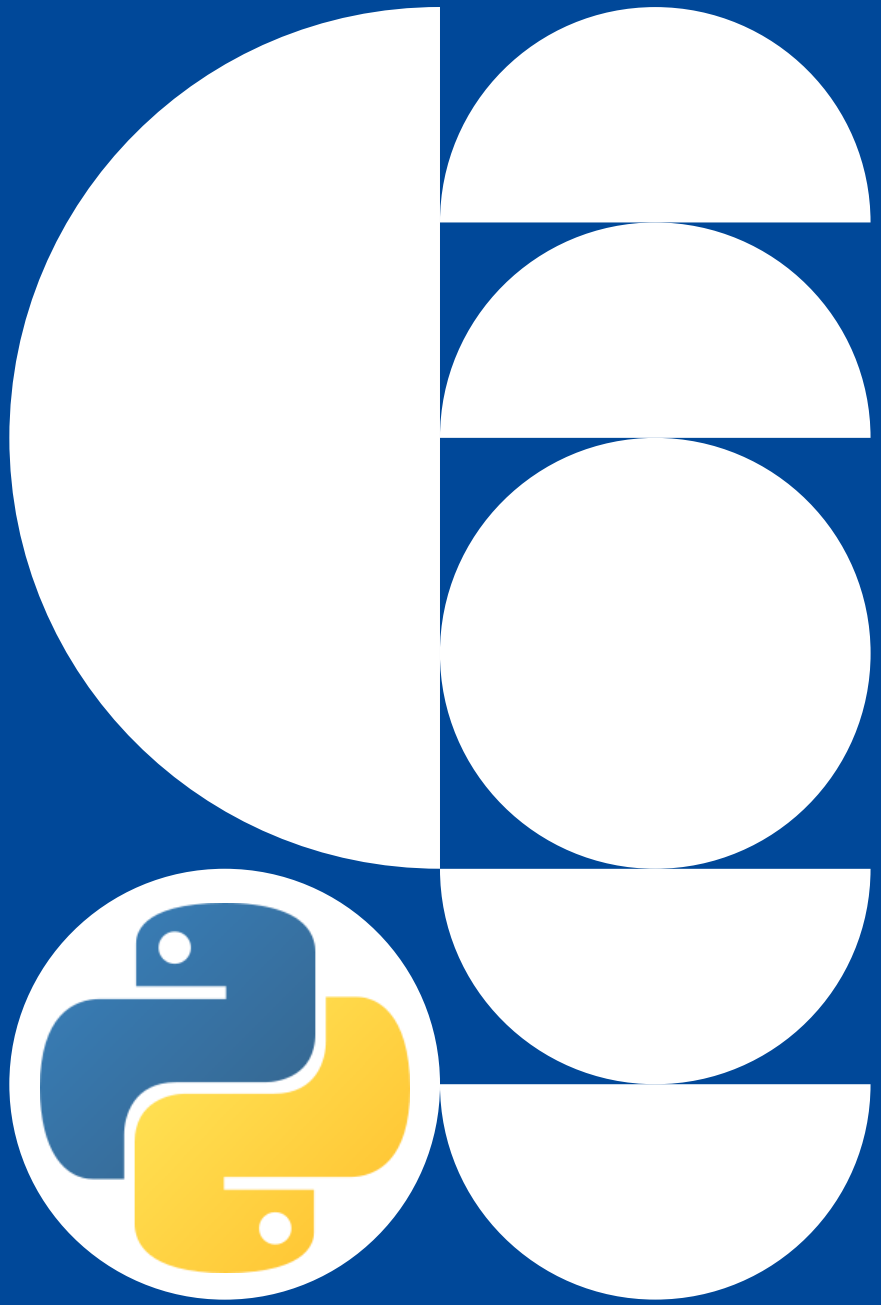


Marburg University

# Programmierpraktikum Einführung in Python

AGs
Becker (Deep Decision Support Systems),
Braun (Natural Language Processing),
Ewerth (Multimodal Modelling and Machine Learning),
Seifert (Explainable Artificial Intelligence)

**Daniel Schneider**

31.08. - 01.09.2026

# Agenda

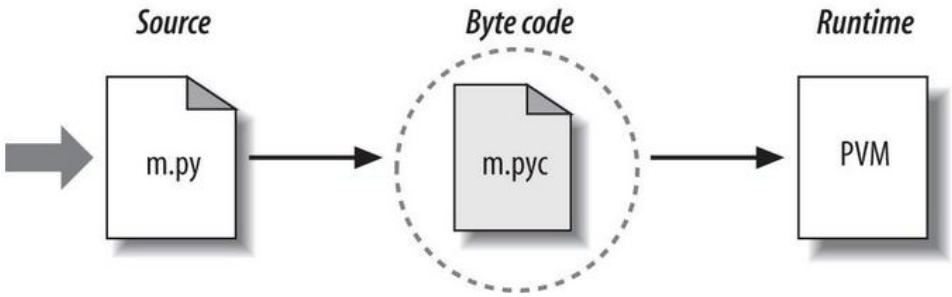
1. Introduction
2. Code Structure
3. Data Types
4. Control Statements
5. Jupyterlab

# Introduction

# What is Python?

- General purpose programming language integrating **procedural**, **functional** and **object-oriented** paradigms
- Named after **Monty Python**
- Aims towards **code readability**, **simplicity** and **generality**
- **Easy to use, Easy to learn** (relatively!)

# Architecture




- **Features**
  - Free, Portable Software
  - Dynamic Typing
  - Garbage Collection
  - Built-In Data Structures (lists, dictionaries)
  - Built-In Standard Operations (join, sort, map)
  - **…**

# Interaktive Python Interpreter

- **Execute commands in an interactive python session**

```
> python                    # start interactive session
Python 3.6.9 (default, Oct 8 2020, 12:12:24)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for
more information.

>>> print("hello world")
hello world

>>> import sys              # module import
>>> print(sys.platform)     # print platform string
linux
>>> hello = 'hello world! '
>>> print(hello * 4)        # string repetition
hello world! hello world! hello world! hello world!

>>> exit()                  # close interactive session
```

- **Run a program executing the same commands**

```
> python helloworld.py      # execute code from file
hello world
linux
hello world! hello world! hello world! hello world!
```

# Code Structure

# Structuring with Indentation

```
def faculty(number):
....if number == 0:
........return 1
....result = number * faculty(number - 1)
....return result
```

- **No bracketing** for code structuring
- Indentation of blocks with whitespaces
  - Convention: indentation depth of **4 spaces**
  - **No tabs!**

# Coding Styles / Naming conventions

- Uniform coding as far as possible

```
import this                                 # only one import per line
from that import object1, object2           # avoid importing everything (*)

UPPERCASE_CONSTANT_NAME = "42"              # constant names in uppercase

class CamelCaseClassName(object):           # class names in camel case
    def underscore_method_name(self):       # method names in lower case with underscores
        pass                                # indent 4 spaces, no tabs!

def format_article():                       # use descriptive identifiers
    is_public = False                       # boolean: is
    has_content = True                      # boolean: has
    text = 'a Text'                         # one element: singular
    texts = []                              # multiple elements: plural
```

- **Style Guide** for Python Code: http://www.python.org/dev/peps/pep-0008/

# Data types

# Data types

| Type | Example | Notes |
|---|---|---|
| Integer (`int`) | `42` | |
| Float (`float`) | `42.0` | |
| String (`str`) | `"python"` | No separate datatype for single characters |
| List (`list`) | `[42, "python", 42.0]` | |
| Tuple (`tuple`) | `(42, "python", 42.0)` | No item assignment after initialization |
| Set (`set`) | `{42, "python"}` | Automatically removes duplicates |
| Dictionary (`dict`) | `{"one": 1, 2: "two"}` | Key-Value Pairs |

# Object Orientation

- **Everything is an object**
- All data objects are **subclasses** of the class ***object***

```
type(1)
```
```
<class 'int'>
```
```
type("hi")
```
```
<class 'str'>
```
```
type(faculty)
```
```
<class 'function'>
```
```
issubclass(type(1), object)
```
```
True
```

- **Functions** are furthermore ***callable***

```
callable(faculty)
```
```
True
```
```
callable(1)
```
```
False
```

# Dynamic Typing

- **No type checking of the compiler**
- Type Checking and errors **at runtime**

```
myvar = "foobar"
type(myvar)
```

```
<class 'str'>
```

Not declared, just assigned!
(no `def`, `string`, `int`, `void` etc.)

```
len(myvar)
```

```
6
```

```
myvar = 4
type(myvar)
```

```
<class 'int'>
```

```
len(myvar)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'int' has no len
```

# Lists

```
list_1 = [0, 1, 2, 3, 4]
list_1[0]
```

```
0
```

```
list_1[-1]
```

Negative values
index from the end

```
4
```

```
list_1[1:3]
```

Slice interval [1, 3)

```
[1, 2]
```

```
list_1[:3]
```

```
[0, 1, 2]
```

```
list_1[3:]
```

```
[3, 4]
```

```
list_1[:]
```

```
[0, 1, 2, 3, 4]
```

# Lists (2)

```
list_1 = list(range(10))
list_1
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
list_1.append(10)
```

```
list_1[4::2]
```

Slice with stepsize 2

```
[4, 6, 8, 10]
```

```
list_2 = [4, "five", faculty, list_1]
list_2
```

Mix of datatypes allowed

```
[4, 'five', <function faculty at 0x14846e>, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]
```

```
list_2[-1][-1]
```

```
9
```

```
list_2[2](6)
```

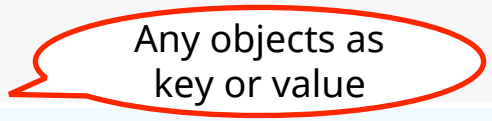
```
720
```

# Dictionaries

```
dict1 = {"a": 1, "b": 2}
dict1
```

```
{'b': 2, 'a': 1}
```

```
dict1["list"] = list1
dict1
```




```
{'b': 2, 'a': 1, 'list': [0, 1, 2, 3]}
```

```
dict1[faculty] = 42
dict1
```

```
{'b': 2, <function faculty at 0x000340F4>: 42, 'a': 1, 'list': [0, 1, 2]}
```

```
dict1[faculty]
```

```
42
```

```
dict1["list"][0]
```

```
0
```

# Strings

```
text = 'This is a string.`
text[0]
```

```
'T'
```

```
text[-1]
```

```
'.'
```

```
text[-7:-1]
```

```
'string'
```

```
text.split()
```

```
['This', 'is', 'a', 'string.']
```

```
text.split("string")
```

```
['This is a ', '.']
```

# Strings (2)

```
first_name = input("First Name:")
```
```
Peter
```
```
last_name = input("Last Name:")
```
```
Müller
```
```
print(f"My name is {first_name} {last_name}")
```
```
My name is Peter Müller
```
```
number_of_es = (first_name + last_name).count("e")
print(f"My name contains {number_of_es:d} e's")
```
```
My name contains 3 e's
```

# Data objects – Boolean Context

| Data object | boolean value | singleton |
|---|---|---|
| `True` | True | ja |
| `False` | False | ja |
| `None` | False | ja |
| `0` | False | |
| `0.0` | False | |
| `1` | True | |
| `[]` | False | |
| `{}` | False | |
| `""` | False | |
| `" "` | True | |

```
bool(0)
```

```
False
```

```
bool(-42)
```

```
True
```

```
if [1,2,3]:
    print("List not empty")
```

```
List not empty
```

# Control Statements

# Control Statements

```
a = 40
if a == 42:
    print("The answer to all questions")
elif a == 41:    # Equality
    print("Close miss")
elif a is 43:    # Identity
    pass
else:
    print("Unfortunately completely wrong")
```

do nothing (needed for indentation)

```
Unfortunately completely wrong
```

- **Inline If**

```
b = 1 if 42 > a else 0
b
```

```
1
```

- There is **no switch/case**

# For-Loops

```
for item in [1, 2]:
    print(item)
```

```
1
2
```

```
for char in "ab":
    print(char)
```

```
a
b
```

```
for key in {"x": 1, "y": 2}:
    print(key)
```

```
x
y
```

```
for key, value in {"x": 1, "y": 2}.items():
    print(key, "->", value)
```

```
X -> 1
Y -> 2
```

# For-Loops (2)

Iterates over `n` from [0, 10)

```
for n in range(10):
    if not n % 2:
        continue
    if n > 5:
        break
    print(n)
```

Forces the next loop cycle

Terminates the loop

```
1
3
5
```

don't care

```
import random
for _ in range(2):
    print(random.random())
```

```
0.20469848981625016
0.7813379886334668
```

```
for i, item in enumerate(["first item", "second item"]):
    print(i, "->", item)
```

```
0 -> first item
1 -> second item
```

# Imperative vs. Functional Approach

- **Example:** Fill that empty list with a list of squares from $0^2$ to $9^2$.

```
squares = []
```

- **Imperative Approach**

```
for x in range(10):
    squares.append(x**2)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- **Functional Approach**

```
squares = list(map(lambda x: x**2, range(10)))
```

Similar to functional languages like e.g. Haskell

- **List Comprehensions**

```
squares = [x**2 for x in range(10)]
```

"Syntactic sugar“ for the imperative approach

# List Comprehensions

```
[faculty(x) for x in range(4)]
```

```
[1, 1, 2, 4]
```

```
[faculty(x) for x in range(4) if x % 2 == 0]
```

```
[1, 2]
```

```
{round(x) for x in [1.3, 1.4, 1.5]}
```

```
{1, 2}
```

Set datatype
removes duplicates

```
{n:n for n in range(5)}
```

```
{0:0, 1:1, 2:2, 3:3, 4:4}
```

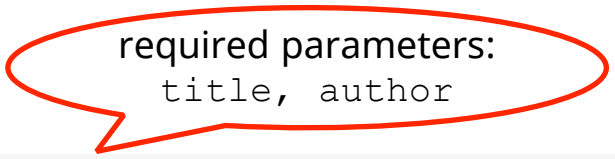
```
words = 'The quick brown fox jumps over the lazy dog'.split()
[(w.upper(), len(w)) for w in words]
```

```
[('THE', 3), ('QUICK', 5), ('BROWN', 5), ('FOX', 3), ('JUMPS', 5),
...]
```

```
list(map(lambda w: (w.upper(), len(w)), words))
```

# Functions and Classes

# Functions




```
def booktitle(title, author):
    print(f"Author: {author}\nTitle: {title}")
```

```
booktitle('Solaris', 'Lem')
```

```
Author: Lem
Title: Solaris
```

```
booktitle(author='Tolkien', title='The Hobbit')
```

```
Author: Tolkien
Title: The Hobbit
```

```
booktitle('Solaris', author='Lem')
```

```
Author: Lem
Title: Solaris
```

```
booktitle(author='Lem', 'Solaris')
```

```
File "<stdin>", line 1
SyntaxError: non-keyword arg after keyword arg
```

# Functions (2)

Default value

```
def booktitle(title, author='unknown'):
    print(f"Author: {author}\nTitle: {title}")

booktitle('Der Schwarm')
```

```
Author: unknown
Title: Der Schwarm
```

position-based arguments

```
def func(*args):
    print(args)

func(1, 2, 42, 'four')
```

```
[1, 2, 42, 'four']
```

keyword arguments

```
def func(**kwargs):
    print(kwargs)

func(author='Lem', title='Solaris')
```

```
{'title': 'Solaris', 'author': 'Lem'}
```

# Classes

Reference to class instance

```
class Book:
    def __init__(self, author = '', title = ''):
        self.author = author
        self.title = title
```

Instance variables

```
book = Book('Bishop', 'Machine Learning')
book.title
```

```
'Machine Learning'
```

```
book2 = Book()
book2.author = 'Bishop'
book2.title = 'Machine Learning'
book2.title
```

```
'Machine Learning'
```

# Classes (2)

```
class Book:
    text_type = "book"

    def __init__(self, author = '', title = ''):
        self.author = author
        self.title = title

    def info(self):
        return self.author, self.title, self.text_type
```

Class variable

Instance variables

```
book = Book('Bishop', 'Machine Learning')
author, title, text_type = book.info()
print("Author: {}  Title: {}".format(author, title))
```

```
Author: Bishop  Title: Machine Learning
```

# Inheritance

```
class PrintableBook(Book):
    def __str__(self):
        return f"Author: {self.author}  Title: {self.title}"

    def info(self, snippet):
        author, title = super().info()
        return f"Author: {self.author}  Title: {self.title}  '{snippet}'"
```

```
book = PrintableBook('Bishop', 'Machine Learning')
print(book)
```

```
Author: Bishop  Title: Machine Learning
```

Implicitely
`print(str(book))`

```
print(book.info("This book is about..."))
```

```
Author: Bishop  Title: Machine Learning  This book is about...
```

# DUCK-TYPING

– *"When I see a bird that walks like a duck and swims like a duck and quacks like a duck, I call that bird a duck."* – James Whitcomb Riley

```
class Duck:
    def quack(self):
        print("Quack, quack!")

    def fly(self):
        print("Flap, Flap!")

class Person:
    def quack(self):
        print("I'm Quackin'!")

    def fly(self):
        print("I'm Flyin'!")
```

```
def in_the_forest(duck):
    duck.quack()
    duck.fly()
```

no interface or abstract class

```
in_the_forest(Duck())
```

```
Quack, quack!
Flap, Flap!
```

```
in_the_forest(Person())
```

```
I'm Quackin'!
I'm Flyin'!
```

# Packages

# Packages

- Package management programs
  - Pip (pre installed with Python)
    - `pip install numpy`
  - Anaconda (Python distribution for scientific computing)
    - `conda install numpy`
- Important Packages:
  - math
  - NumPy (Extended Math Functionality with focus on Matrices)
  - PIL (Image Functionality)
  - matplotlib (Plots)
  - Pandas (Dataframes)
  - Scikit Learn (Machine Learning)

# Numpy

# Numpy

- Core library for scientific computing in Python
- High-performance multidimensional arrays
- Faster than standard Python
- Large collection of high-level mathematical functions
- Open-source software
- Many contributors

# Numpy vs. Core Python

```
import numpy as np
```

```
cvalues = [25.3, 24.8, 26.9, 23.9]
print(cvalues)
```

```
[25.3, 24.8, 26.9, 23.9]
```

```
C = np.array(cvalues)
print(C)
```

```
[25.3, 24.8, 26.9, 23.9]
```

```
print(C * 9 / 5 + 32)
```

Convert to degrees Fahrenheit

```
[77.54, 76.64, 80.42, 75.02]
```

```
fvalues = [x*9/5 + 32 for x in cvalues]
print(fvalues)
```

Standard Python is awkward here

```
[77.54, 76.64, 80.42, 75.02]
```

# Numpy Arrays

- Grid of values of **same type**
- Indexed by tuple of nonnegative integers
- **Rank**: number of dimensions
- **Shape**: Tuple giving the size along each dimension

```
a = np.array([1, 2, 3])
```

Create rank 1 array

```
type(a)
```

```
<type 'numpy.ndarray'>
```

```
a.shape
```

```
(3,)
```

Create rank 2 array

```
b = np.array([[1,2,3],[4,5,6]])
b.shape
```

```
(2,3)
```

# Array Indexing

```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
```

Which rank and shape?

```
a[0, 1]
```

```
2
```

```
a[0, [1,2]]
```

```
[2 3]
```

```
b = a[:2, 1:3]
b
```

Slicing similar to lists

```
[[2 3]
 [6 7]]
```

```
b[0, 0] = 77
a[0, 1]
```

Slice is a view

```
77
```

# Boolean Array Indexing

```
a = np.array([[1, 2], [3, 4], [5, 6]])
```

```
bool_idx = (a > 2)
bool_idx
```

```
[[False False]
 [ True  True]
 [ True  True]]
```

```
a[bool_idx]
```

```
[3 4 5 6]
```

```
a[a > 2]
```

```
[3 4 5 6]
```

# Broadcasting

- Work with arrays of different shapes when performing arithmetic operations
- E.g. perform some operation on a larger array by using smaller array multiple times

- Example with standard python:

```
x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
v = np.array([1, 0, 1])
```

```
y = np.empty_like(x)
for i in range(4):
    y[i, :] = x[i, :] + v
print(y)
```

Add the vector v to each row

```
[[ 2  2  4]
 [ 5  5  7]
 [ 8  8 10]
 [11 11 13]]
```

# Broadcasting (2)

- Explicit for-Loop can be slow for large arrays!
  → Solution: Broadcasting

```
x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
v = np.array([1, 0, 1])
```

```
y = x + v
print(y)
```

```
[[ 2  2  4]
 [ 5  5  7]
 [ 8  8 10]
 [11 11 13]]
```

- v is copied as often needed to fill shape of x, sum elementwise

# Broadcasting (3)

Broadcasting follows these rules:

1. If arrays do not have same rank: prepend shape of lower rank arrays with 1s until both have same length
2. Two arrays are *compatible* in a dimension if they have the same size in this dimension or one of the arrays has size 1 in this dimension
3. Arrays can be broadcast together if they are compatible in all dimensions
4. After broadcasting, each array behaves as if it had shape equal to the elementwise maximum of shapes of the two input arrays
5. In any dimension where one array had size 1 and the other array had size >1, the first array behaves as if it were copied along that dimension

# Pandas

# Pandas

- Library for data analysis and data manipulation

- Name is derived from „**Pan**el **Da**ta“ (multidimensional data)

- Features:
  - Fast, efficient implemtation of DataFrames
  - Load data sets in various file formats
  - Data alignment and handling of missing values
  - Easily slice, index, and split large data sets
  - Data aggregation and transformation

# Data Structures

- **Series**
  - 1D-Array
  - Values of an attribute (a column) from a dataset
  - Index labels for referencing the values (rows)
  - Homogeneous data (all values have the same data type)

- **DataFrame**
  - 2D table structure
  - Data set with multiple attributes (series)
  - Index labels for referencing rows
  - Attribute labels for referencing columns
  - Heterogeneous data (but each column is homogeneous)

# DataFrame - Creation

```
import pandas as pd
```

```
data = [['Alex',10],['Bob',12],['Clarke',13]]
pd.DataFrame(data, columns=['Name','Age'])
```

Row-wise data as lists

```
     Name  Age
0    Alex   10
1     Bob   12
2  Clarke   13
```

Set column names

```
data_dict = {'Name':['Alex','Bob','Clarke'], 'Age':[10,12,13]}
pd.DataFrame(data, index=[100,101,102])
```

Column-wise data as dictionary

```
       Name  Age
100    Alex   10
101     Bob   12
102  Clarke   13
```

```
s1 = pd.Series({100: 'Alex', 101: 'Bob', 102: 'Clarke'})
s2 = pd.Series({100: 10, 102: 13})
pd.DataFrame({'Name': s1, 'Age': s2})
```

Column data as Series

Data as dictionary of series

```
       Name   Age
100    Alex  10.0
101     Bob   NaN
102  Clarke  13.0
```

Missing value as `NaN`

# DataFrame - Indexing

```
data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data, columns=['Name','Age'], index=[100,101,102])
```

```
df['Name']
```

Access column via attribute label

```
100      Alex
101       Bob
102    Clarke
Name: Name, dtype: object
```

```
df.loc[[100,102]]
```

Access rows via list of index labels

```
       Name  Age
100    Alex   10
102  Clarke   13
```

```
df.loc[[100,102],'Name']
```

Access rows and columns via index and attribute labels

```
100      Alex
102    Clarke
```

```
df.iloc[[0,2],0]
```

Access to rows at index 0 and 2 as well as colum at index 0

```
100      Alex
102    Clarke
```

# Data analysis

Load a DataFrame from a csv file

```
df2 = pd.read_csv(input_path, sep=',', header='infer', index_col='Id')
```

Extract column names from the file

Set index column

Output number of rows and columns

```
df2.shape
```

```
(150, 3)
```

Output data types of the columns

```
df2.dtypes
```

```
sepal_length    float64
petal_length    float64
species          object
dtype: object
```

Output the first n rows

```
df2.head(4)
```

```
   sepal_length  petal_length  species
0           5.1                   1.4   setosa
1           4.9           1.4   setosa
2           4.7           1.3   setosa
3           4.6                   1.5   setosa
```

# Descriptive statistics

Calculate base statistics for all columns

```
df2.describe(include='all')
```

```
       sepal_length  petal_length     species
count    150.000000    150.000000         150
unique          NaN           NaN           3
top             NaN           NaN  versicolor
freq            NaN           NaN          50
mean       5.843333      3.758000         NaN
std        0.828066      1.765298         NaN
min        4.300000      1.000000         NaN
25%        5.100000      1.600000         NaN
50%        5.800000      4.350000         NaN
75%        6.400000      5.100000         NaN
max        7.900000      6.900000         NaN
```

Different statistics for numerical and categorical attributes, rest is filled with `NaN`

```
df2[['sepal_length','petal_length']].sum()
```

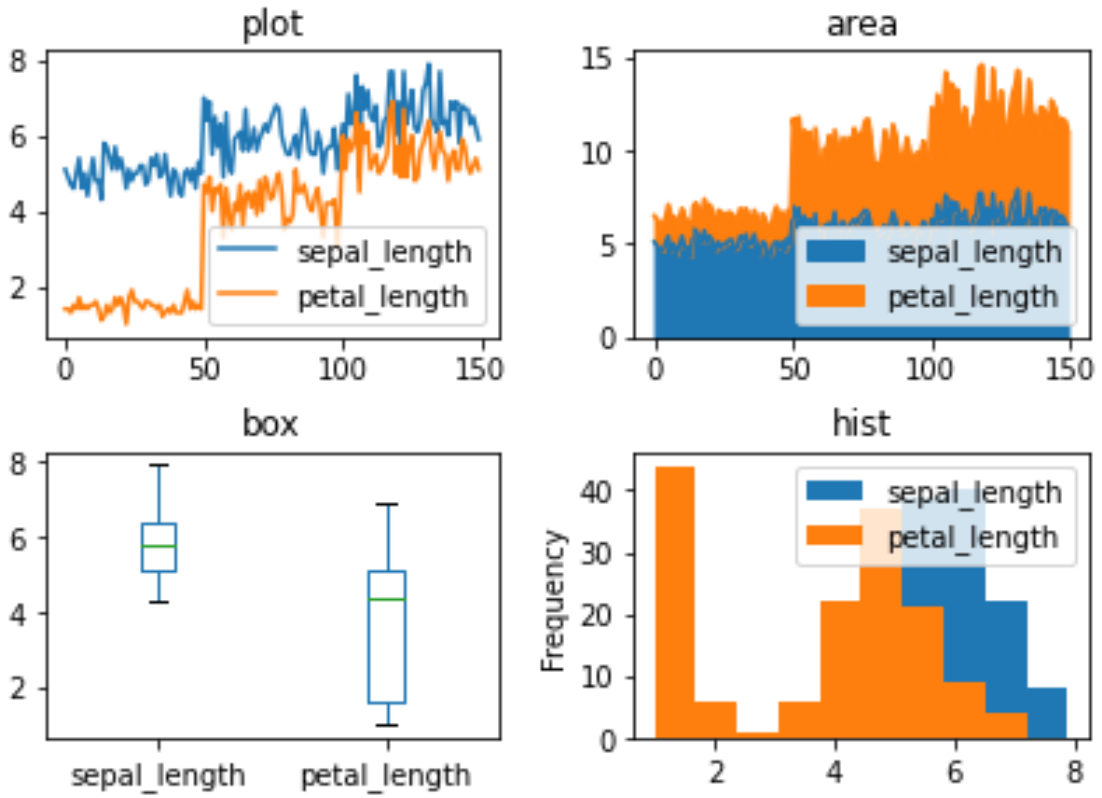
```
sepal_length   876.5
petal_length   563.7
dtype: object
```

Manually calculate statistics (e.g., sum over columns)

# Matplotlib

# Matplotlib

- Library for data visualization
- Pyplot module for creation of 2D-Plots
- Functionality similar to MATLAB
- Support for many different plot types
  - Line Plots, Scatterplots, Area Plots
  - Pie Charts, Bar Plots
  - Histograms, Boxplots
  - …



# Simple Plot

```
import matplotlib.pyplot as plt

import numpy as np
import math
x = np.arange(0, math.pi*2, 0.05)
y = np.sin(x)
```

Calculate values of a sine function

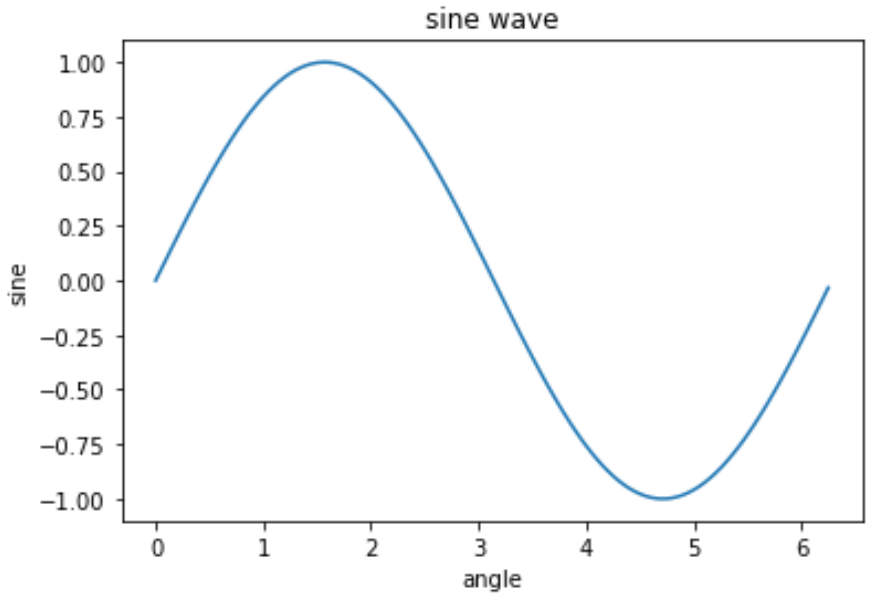
- Create a line plot

```
plt.plot(x,y)

plt.xlabel("angle")
plt.ylabel("sine")
plt.title('sine wave')

plt.show()
```

Create a simple line plot of values (x,y)

Set axis names and plot title for the current plot

Show the current plot

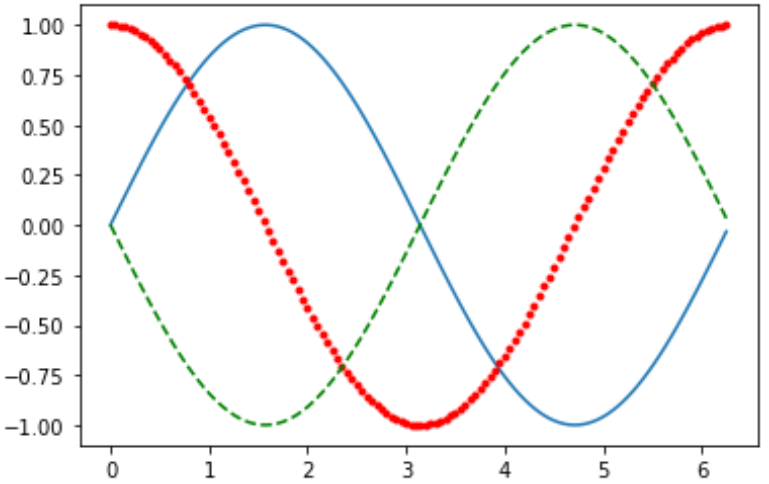


# Simple Plot (2)

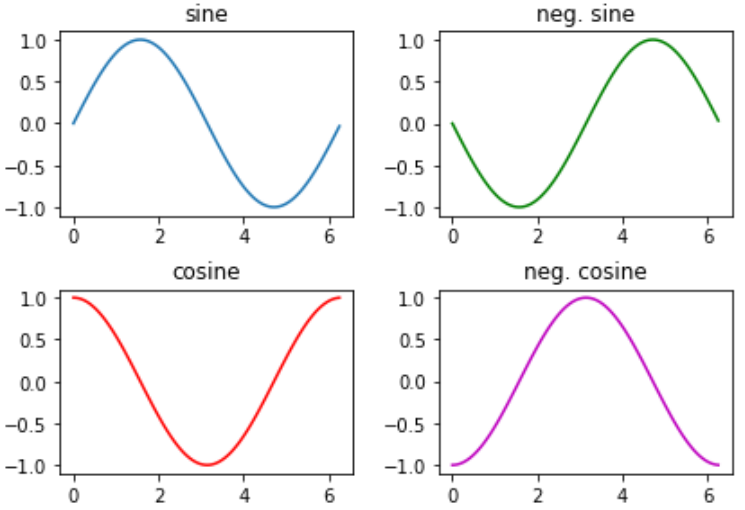
- As long as `plt.show()` has not been called, all plots are added to one figure

```
plt.plot(x, np.sin(x), '-')
plt.plot(x, np.cos(x), 'r.')
plt.plot(x, -np.sin(x), 'g--')
plt.show()
```

Set color and line style




- Figures can be used to create more complex plots (e.g., consisting of multiple graphs)



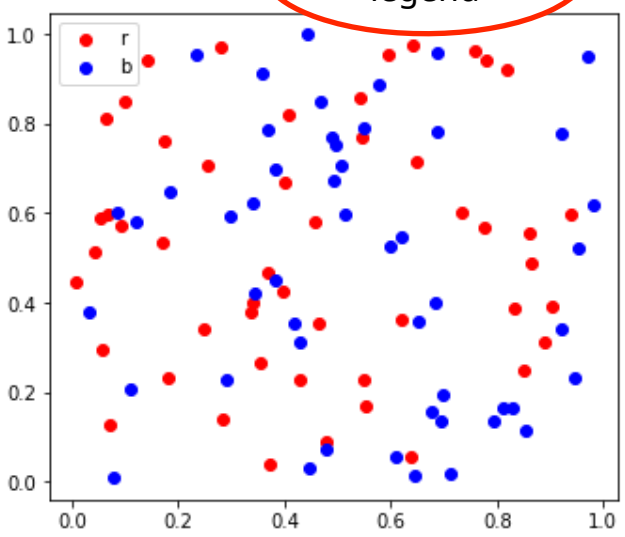
# Plot-Types

## – Scatterplot

```
x = np.random.rand(2,50)
y = np.random.rand(2,50)

plt.scatter(x[0], y[0], color='r', label='r')
plt.scatter(x[1], y[1], color='b', label='b')
plt.legend()
plt.show()
```

Create a legend

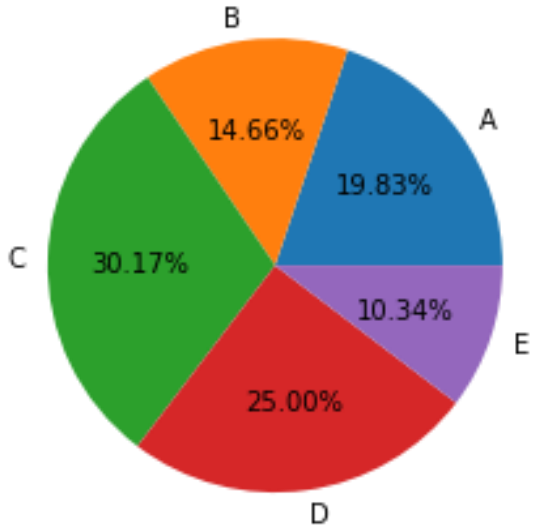
Assign labels to the point rows




## – Pie Chart

```
options = ['A', 'B', 'C', 'D', 'E']
amount = [23, 17, 35, 29, 12]

plt.pie(amount, labels=options, autopct='%.2f%%')

plt.show()
```

Format percentage values



# Jupyterlab

# iPython, Jupyter Lab

- Web based interface
- Uses the power of modern web technologies
- Mix code with description (markdown)
- Stores notebook in a JSON file .ipynb

# Jupyter Lab

- Installation

```
> pip install jupyterlab
```

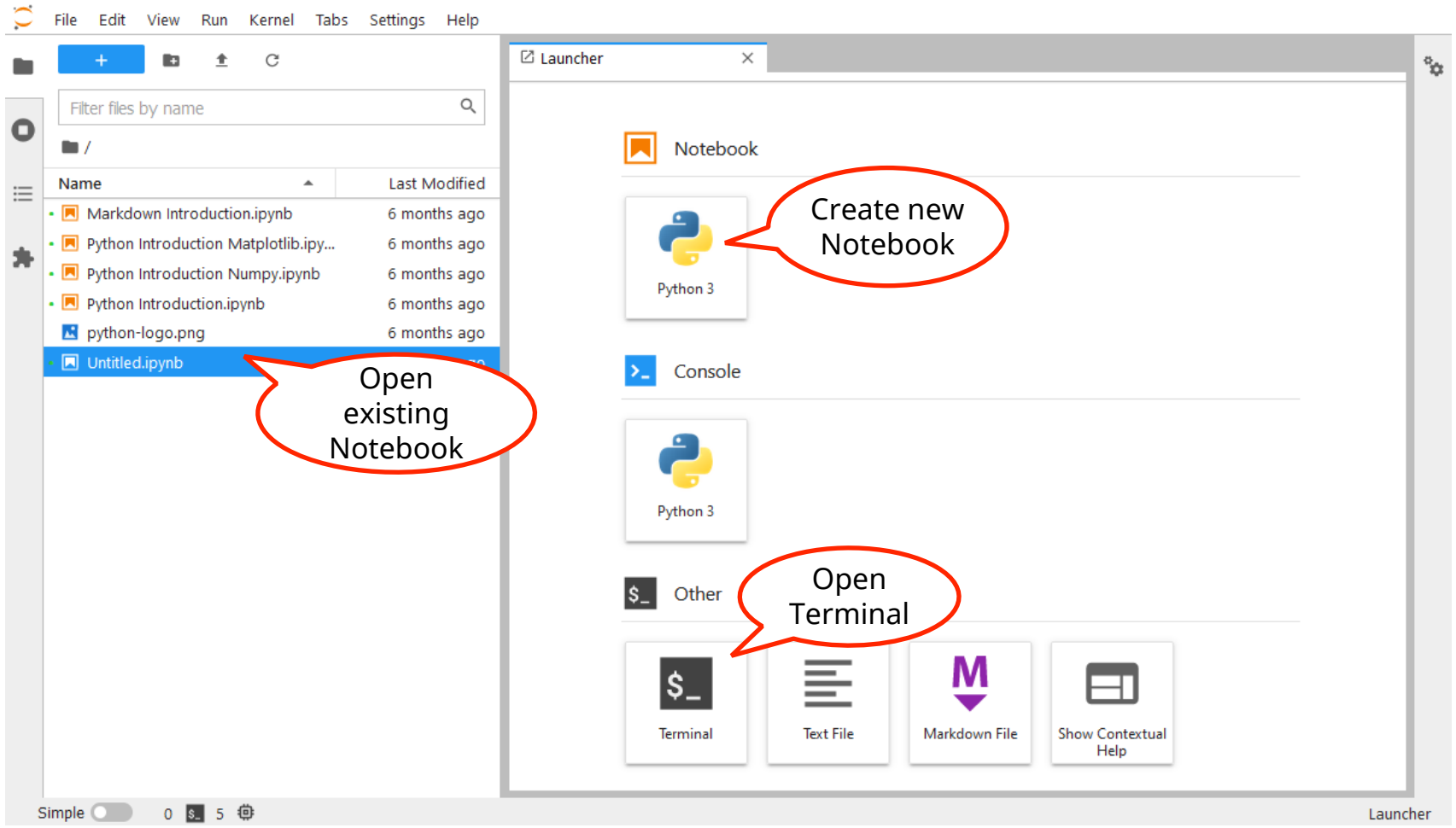
- Start Jupyter Lab

```
<navigate to desired directory>

> jupyter lab
```

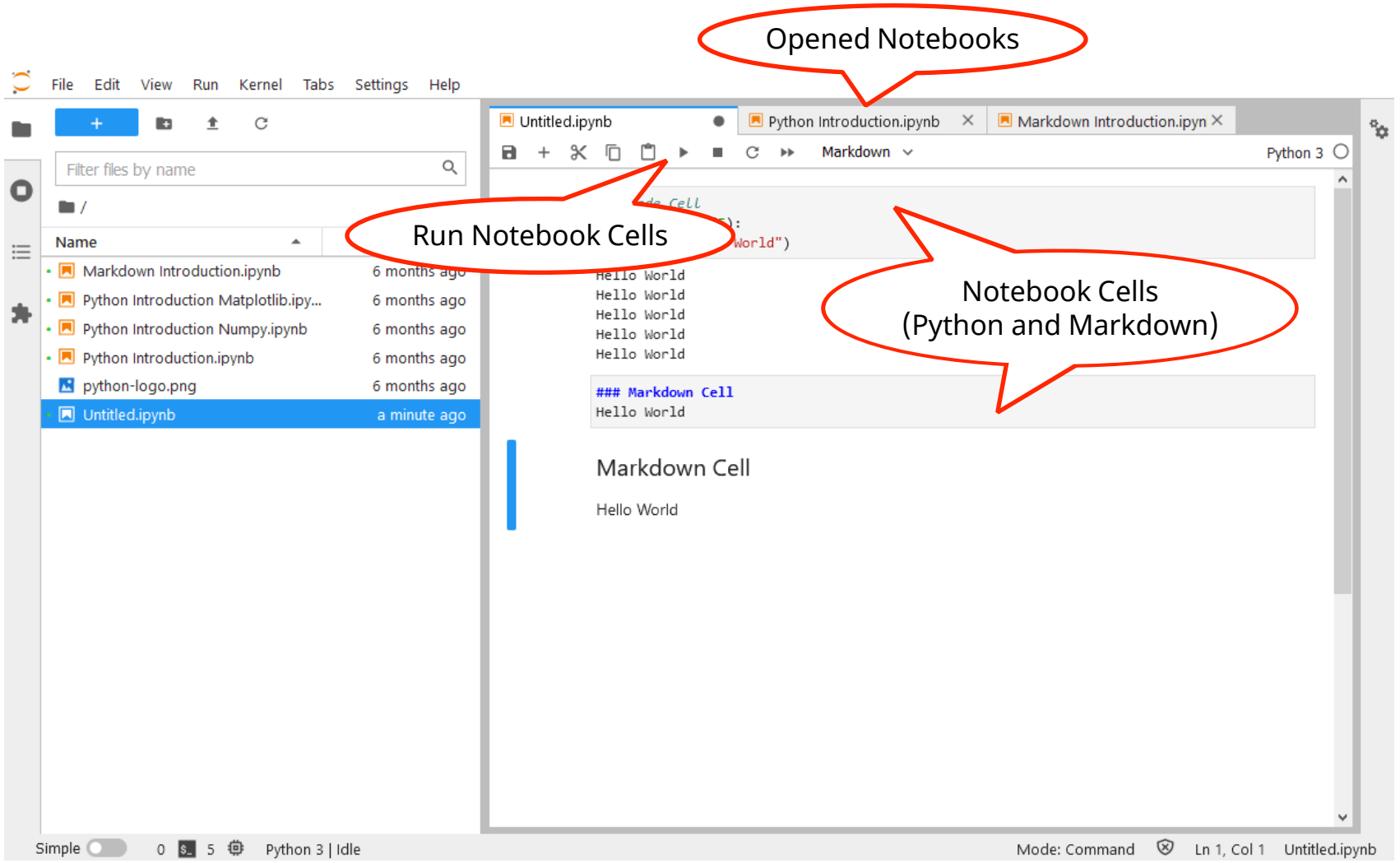
```
...
[I 12:29:01.381 ServerApp] Jupyter Server 1.6.4 is running at:
[I 12:29:01.381 ServerApp] http://localhost:8888
[I 12:29:01.381 ServerApp] Use Control-C to stop this server and shut down
all kernels (twice to skip confirmation).
...
```

- Open http://localhost:8888 in a web browser

# Jupyter Lab Layout



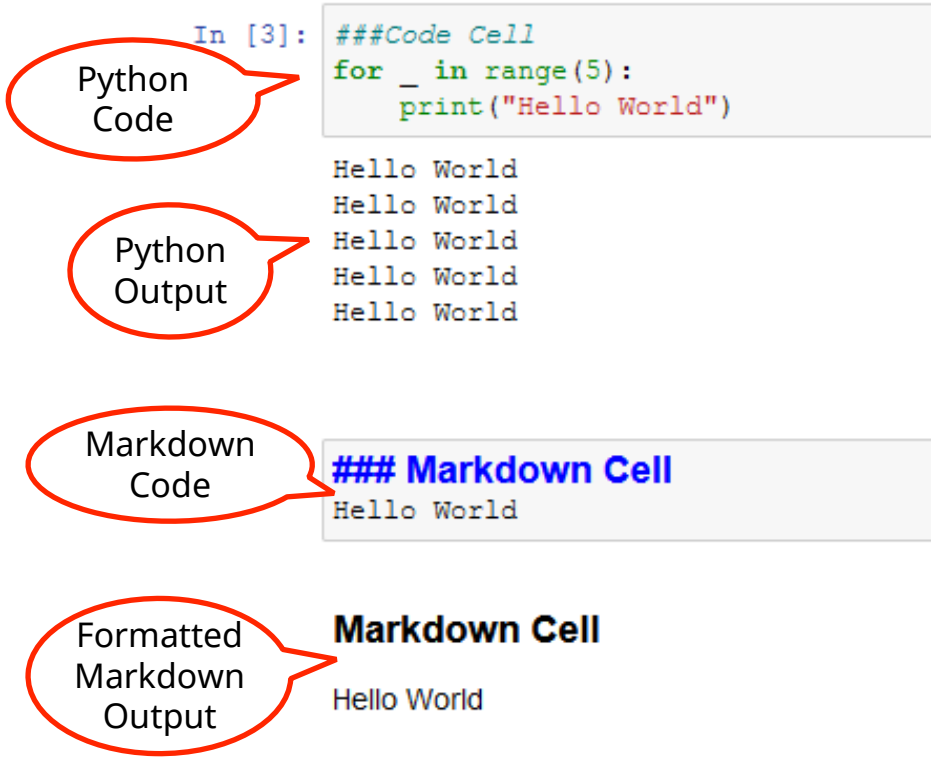
# Jupyter Lab Layout (2)



# Notebook Cells

- **Python Cells**
  - Write Python statements
  - When the cell is run, the result is displayed in an output cell

- **Markdown Cells**
  - Write text in markdown language
  - When the cell is run, the formatted output is displayed



# Keyboard Shortcuts

- **CTRL + ENTER**: Run the selected cells

- **M**: Make the selected cells Markdown Cells
- **Y**: Make the selected cells Python Cells

- **B**: Add a new cell below the selected cell
- **A**: Add a new cell above the selected cell
- **D,D**: Delete the selected cell

- **C**: Copy the selected cell
- **X**: Cut the selected cell
- **V**: Paste copied cells below the selected cell
- **SHIFT + V**: Paste copied cells above the selected cell

# Magic Commands

- IPython enhancements to the standard Python shell

- Line Magics: Operate on single line

```
%time f = faculty(1000)
```

Measure the execution time of a single command

```
Wall time: 499 µs
```

```
%who function  # str, int, ...
```

List all variables (here of type function)

```
faculty
```

```
%cd "../../"
```

Change working directory

```
F:\Dokumente\Uni-Projekte\Vorlesungen
```

```
%pinfo faculty
```

Print documentation for variables

```
Signature: faculty(number)
Docstring: <no docstring>
Type:      function
```

# Magic Commands (2)

- IPython enhancements to the standard Python shell
- Cell Magics: Operate on whole code block

```
%%time
f = faculty(1000)
```

Measure the execution time of whole code block

```
Wall time: 499 µs
```

- Command overview

```
%lsmagic
```

# Next Steps

# Next Steps

- Install Python from https://www.python.org/
- Install Jupyterlab via pip

```
> pip install jupyterlab
```

- Download the files from https://ilias.uni-marburg.de/goto.php/fold/4776269
- Navigate to the folder containing the downloaded files in a terminal
- Start jupyter lab in this folder

```
> jupyter lab
```

- Open http://localhost:8888 in a web browser if it does not open automatically
- The tasks for the first day are found in the notebook **`ProPra_AI_day1.ipynb`**
- Have fun!

- If you run into problems, ask the tutors for help